

```
"""Populate the shipment database from the supplied shipping CSV files."""
```

```
import csv
import sqlite3
from collections import Counter
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"
DATABASE_PATH = BASE_DIR / "shipment_database.db"
```

```
def get_product_id(connection, product_name):
    connection.execute(
        "INSERT OR IGNORE INTO product (name) VALUES (?)",
        (product_name,),
    )

    product_row = connection.execute(
        "SELECT id FROM product WHERE name = ?",
        (product_name,),
    ).fetchone()

    return product_row[0]
```

```
def load_shipment_details():
    csv_path = DATA_DIR / "shipping_data_2.csv"
    shipment_details = {}

    with csv_path.open(newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            shipment_id = row["shipment_identifier"]

            shipment_details[shipment_id] = {
                "origin": row["origin_warehouse"],
                "destination": row["destination_store"],
            }

    return shipment_details
```

```
def import_shipping_data_0(connection):
    csv_path = DATA_DIR / "shipping_data_0.csv"
    inserted_shipments = 0

    with csv_path.open(newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)
```

```

for row in reader:
    product_id = get_product_id(connection, row["product"])

    connection.execute(
        """
        INSERT INTO shipment
            (product_id, quantity, origin, destination)
        VALUES (?, ?, ?, ?)
        """,
        (
            product_id,
            int(row["product_quantity"]),
            row["origin_warehouse"],
            row["destination_store"],
        ),
    )

    inserted_shipments += 1

return inserted_shipments


def count_products_by_shipment():
    csv_path = DATA_DIR / "shipping_data_1.csv"
    product_counts = Counter()

    with csv_path.open(newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            group_key = (
                row["shipment_identifier"],
                row["product"],
            )
            product_counts[group_key] += 1

    return product_counts


def import_shipping_data_1_and_2(connection):
    shipment_details = load_shipment_details()
    product_counts = count_products_by_shipment()
    inserted_shipments = 0

    for group_key, quantity in product_counts.items():
        shipment_id, product_name = group_key
        details = shipment_details[shipment_id]
        product_id = get_product_id(connection, product_name)

        connection.execute(

```

```

        """
        INSERT INTO shipment
            (product_id, quantity, origin, destination)
        VALUES (?, ?, ?, ?)
        """
        (
            product_id,
            quantity,
            details["origin"],
            details["destination"],
        ),
    )

    inserted_shipments += 1

return inserted_shipments

def main():
    print(f"Database: {DATABASE_PATH}")

    connection = sqlite3.connect(DATABASE_PATH)

    try:
        connection.execute("PRAGMA foreign_keys = ON")

        foreign_keys_enabled = connection.execute(
            "PRAGMA foreign_keys"
        ).fetchone()[0]

        if foreign_keys_enabled != 1:
            raise RuntimeError("SQLite foreign keys are not enabled")

        with connection:
            imported_from_0 = import_shipping_data_0(connection)
            imported_from_1_and_2 = import_shipping_data_1_and_2(
                connection
            )
    finally:
        connection.close()

    print(
        f"Imported {imported_from_0} shipments "
        f"from shipping_data_0.csv"
    )
    print(
        f"Imported {imported_from_1_and_2} shipments "
        f"from shipping_data_1.csv and shipping_data_2.csv"
    )

```

```
if __name__ == "__main__":  
    main()
```